

RL-Based Mixed-Criticality RTOS Scheduler: From PPO Training to Bare-Metal Deployment

Ojasw Kant

Department of Computer Science and Engineering
Shiv Nadar Institute of Eminence, Delhi NCR, India

Abstract—Classical real-time schedulers such as EDF (Earliest Deadline First) and RMS (Rate Monotonic Scheduling) are provably optimal for uniprocessor workloads with uniform criticality and utilization ≤ 1 , but degrade systematically under two realistic conditions: *mixed criticality*, where tasks carry different safety importance, and *overload* ($U > 1$), where deadline misses are unavoidable and the scheduler must choose which tasks to sacrifice. We train a PPO reinforcement learning agent with asymmetric criticality rewards (5 $\times$ penalty for HI-critical misses) using a 3-phase curriculum strategy and deploy the learned policy as a Q10 fixed-point integer neural network inside the timer interrupt handler of two real operating systems: a bare-metal ARM Cortex-M4 RTOS in Rust and MIT’s xv6-riscv teaching OS. On the Vestal (2007) inverted-priority mixed-criticality taskset, the RL policy achieves 0 HI-critical misses versus EDF’s 5 and RMS’s 8 on xv6. On hard overload simulation ($U = 1.87$), the policy achieves 59% fewer critical misses than EDF and 96% fewer than Round Robin. Code footprint is 5.1 KB on ARM and 54 KB in the xv6 kernel including weights, with inference completing in under one scheduler tick.

Index Terms—real-time scheduling, mixed criticality, reinforcement learning, proximal policy optimization, embedded systems, RTOS, Vestal failure mode, fixed-point inference.

I. INTRODUCTION

EDF is provably optimal for deadline misses on a single core when all tasks have equal importance and utilization ≤ 1 [1]. Real embedded systems routinely violate both assumptions.

First, mixed-criticality systems assign different safety levels to tasks: flight attitude control must never miss its deadline; telemetry logging is expendable. EDF treats all deadlines identically and cannot encode this distinction. Second, safety-critical embedded systems are routinely designed with superutilization ($U > 1$) [2]: worst-case execution times are rarely achieved simultaneously, so the system meets its deadlines under normal conditions but the scheduler must make principled choices during worst-case bursts. EDF’s greedy nearest-deadline heuristic has no concept of task importance; it will sacrifice a safety-critical task with a far deadline to serve a soft task with a near deadline.

Vestal (2007) [2] formalized this failure mode: when HI-critical tasks have longer periods than LO-soft tasks (the inverted-priority scenario), EDF perpetually selects LO tasks first (nearer deadlines), starving HI tasks entirely. This is not a pathological edge case—it arises naturally in applications such as drone flight controllers, where safety-critical IMU and PID loops have fixed high-frequency periods while soft telemetry and logging tasks run at shorter burst intervals.

This paper presents an end-to-end system that trains a PPO reinforcement learning agent to act as a mixed-criticality RTOS scheduler and deploys the learned policy as a Q10 fixed-point integer neural network on bare metal. The agent receives asymmetric rewards (5 $\times$ penalty for HI-critical misses) and is trained with a 3-phase curriculum, learning first to respond to urgency signals and then to override them in

favor of criticality. The trained policy is exported as integer arrays and runs identically in a bare-metal ARM Cortex-M4 RTOS (Rust) and MIT’s xv6-riscv OS (C), executing 72 integer multiply-accumulate operations per scheduler tick with no floating-point hardware and no external runtime.

The main contributions are: (1) a Gymnasium-compatible RTOS simulation environment with mixed criticality and curriculum learning; (2) an end-to-end Q10 fixed-point export pipeline for kernel deployment; (3) full integration of five runtime-switchable schedulers into xv6-riscv with new syscalls and starvation prevention; and (4) empirical evaluation across 35 scheduler $\times$ taskset combinations confirming Vestal’s theoretical predictions.

II. RELATED WORK

A. Classical Real-Time Scheduling

Liu & Layland [1] proved EDF optimal for uniprocessor implicit-deadline periodic tasks when $U \leq 1$ and all tasks are equally important. Rate Monotonic Scheduling (RMS) [1] assigns static priorities by period; it is optimal among fixed-priority schedulers with the same constraints. Our results confirm both: on uniform-criticality tasksets with $U < 1$, EDF and RMS achieve 0 misses, matching the NN policy. The contribution of this work lies in the regime where Liu & Layland’s assumptions break.

Vestal [2] introduced the mixed-criticality model and demonstrated that EDF fails to protect HI-critical tasks when their periods are longer than LO-soft tasks under superutilization—the inverted-priority scenario. Our experimental results directly confirm this prediction on both Python simulation and live xv6-riscv execution.

B. RL for Scheduling

DeepRM [6] and Decima [7] apply deep RL to data-center job scheduling, demonstrating that learned policies outperform hand-tuned heuristics for cluster bin-packing and DAG scheduling. However, both targets differ fundamentally from hard real-time embedded systems: they operate at millisecond-to-second decision intervals, use Python/TensorFlow inference, and do not encode task criticality. Our policy must complete inference in under 1 ms on a microcontroller without an FPU; the Q10 fixed-point MLP is purpose-built for this constraint. To our knowledge, this work is the first to deploy a PPO-trained scheduler as Q10 integer arrays inside a kernel interrupt handler.

C. Multi-Level Feedback Queue

MLFQ [8] is the closest classical scheduler to “adaptive” behavior and the default in macOS (BSD). It demotes CPU-bound tasks to lower priority queues and promotes them via aging. We implement MLFQ as mode 4 in xv6 (4 queues, aging every 100 ticks). Our results show a fundamental incompatibility with hard real-time mixed-criticality workloads: any task whose WCET exceeds the Q0 quantum (2 ticks) is demoted on first execution, making MLFQ unable to distinguish a safety-critical sensor loop from a background logger. On the drone benchmark, MLFQ produces 50 HI-critical misses—identical to Round Robin.

III. SYSTEM ARCHITECTURE

The system spans three layers: a Python training environment, a bare-metal ARM Cortex-M4 RTOS in Rust, and the xv6-riscv OS in C. The same Q10 fixed-point weights run on both hardware targets without modification.

A. Python Training Environment

The training environment is a Gymnasium-compatible RTOS simulation. Key design decisions:

- RandomRTOSEnv: fresh random taskset every episode, forcing generalization and preventing period-distribution overfitting.
- 3-phase curriculum: $U = 1.03 \rightarrow 1.57 \rightarrow 1.87$. Phase 1 teaches urgency response; phase 3 teaches criticality override.
- Variable execution: actual exec $\in [WCET/2, WCET]$ per job; NN observes remaining_work.
- Asymmetric reward: HI-critical miss = -5 per episode; LO-soft miss = -1. Completion = +1.
- Multi-objective terms: completions, misses, urgency, starvation penalty, jitter, context-switch cost.

Training uses PPO (stable-baselines3) for 2M timesteps with clipped surrogate objective ($\epsilon = 0.2$). A 108-configuration hyperparameter sweep was conducted over learning rate, entropy coefficient, and network width. PPO was chosen over DQN (avoids value-network export complexity) and A3C (avoids non-deterministic gradient accumulation incompatible with curriculum phase transitions).

B. Bare-Metal ARM Cortex-M4 RTOS

The ARM target is a no_std Rust program for the STM32F411 (emulated in QEMU). Preemption uses ARM exception mechanisms:

- SysTick (1 ms tick): deadline checking, job releases, NN inference, PendSV trigger.
- PendSV: assembly context switch—saves/restores r4–r11, swaps stack pointers.
- Per-task stacks: $6 \times 1 \text{ KB} + \text{idle}$, initialized with Cortex-M exception frames.
- Binary: 5.1 KB code + 6.7 KB RAM. Fits STM32F411 (512 KB flash, 128 KB RAM).

C. xv6-riscv Integration

The xv6 integration adds a three-tier scheduler to MIT’s RISC-V teaching OS: Tier 0 handles RT task cleanup; Tier 1 dispatches via the selected scheduler mode; Tier 2 provides starvation-prevention round-robin for non-RT processes (shell, init). Five new syscalls expose RT functionality: rtregister, rtjobdone, rtstats, setscheduler (modes 0–4), and rtremaining. The scheduler mode is switchable at runtime without reboot. The kernel compiles to 54 KB with Q10 weights baked in; all existing xv6 programs execute unmodified.

D. Neural Network and Deployment

The actor network is a $24 \rightarrow 32 \rightarrow 32 \rightarrow 7$ MLP with ReLU activations. Each of 6 task slots contributes 4 Q10 features: time_to_deadline, time_since_scheduled, remaining_ratio, and urgency_rank, each normalized to $[0, 1024]$. Inference performs 72 integer multiply-accumulate operations per tick in int32_t with saturation on overflow. Weights are exported via export_weights.py, which patches both src/policy.rs (Rust) and kernel/policy.c (C) simultaneously. The critic network is discarded at deployment.

IV. EXPERIMENTAL SETUP

All xv6 experiments run on QEMU qemu-system-riscv64 with CPUS=1. ARM experiments run on qemu-system-arm -machine lm3s6965evb. Python experiments average results over 20 evaluation episodes per scheduler.

TABLE I PYTHON SIMULATION TASKSETS

Taskset	U_{no_m}	Description
Normal	1.03	Baseline; EDF optimal (Liu & Layland regime)
Stressed	1.15	Mild overload
Hard	1.57	Mixed crit., variable exec. times
Very Hard	1.87	Severe overload; main comparison
Vestal MC	1.33	Inverted priority: LO tasks shorter periods than HI tasks

TABLE II XV6 BENCHMARK PROGRAMS

Program	U	Task Bodies
rtbench	1.90	Spin loops; IoT workload (sensor $\rightarrow$ sync)
rtvestal	1.33	Spin loops; Vestal (2007) inverted-priority MC

rt drone	1.5 0	Q10 AHRS filter, PID + discrete plant model
rtmaladalen	1.1 6	Mälardalen WCET benchmarks (matmul, CRC-32, etc.)
rtgui	1.0 4	Real computation: LFSR, BFS, brush render
rtgui_a	1.3 7	Heavier GUI: 7×7 brush, alpha blend ($U_h^1=0.88$)
rtgui_b	1.6 9	Vestal GUI: compositor LO T=12 vs render HI T=16

V. RESULTS

All results report HI-critical and LO-soft deadline miss counts. Lower is better. Bold values indicate the best (or tied-best) scheduler for HI-critical misses.

A. Python Simulation: Very Hard Taskset

Table III shows results on the Very Hard taskset ($U = 1.87$, 3 HI-critical, 3 LO-soft tasks, variable execution times).

TABLE III PYTHON SIMULATION: VERY HARD TASKSET ($U = 1.87$)

Scheduler	HI Misses	LO Misses	Reward
PPO	2.0	16.8	558
RMS	3.4	16.0	525
EDF	4.9	16.7	489
Round Robin	50.8	8.9	-631

PPO reduces HI-critical misses by **59%** versus EDF (2.0 vs 4.9) and 41% versus RMS. The reward differential confirms that the PPO policy is not merely minimizing total misses—it is specifically protecting critical tasks at the expense of soft-task throughput, exactly as the asymmetric training reward intended.

B. Python Simulation: Vestal Inverted-Priority

Table IV shows the canonical Vestal failure mode: HI-critical tasks have periods 3–6× longer than LO-soft tasks; $U_{20} = 1.075 > 1$. EDF's greedy deadline rule perpetually selects LO tasks, starving HI tasks.

TABLE IV PYTHON SIMULATION: VESTAL TASKSET ($U = 1.33$)

Scheduler	HI Misses	LO Misses	Total
PPO	2	127	129
Round Robin	0*	88	88
EDF	7	28	35
RMS	13	16	29

PPO achieves **71% fewer HI-critical misses than EDF** (2 vs 7). Round Robin achieves 0 HI misses coincidentally: $U_h^1 = 0.25$, so equal CPU share (1/6 per task) accidentally covers all HI deadlines. On any taskset where $U_h^1 > 1/n$ this coincidence disappears.

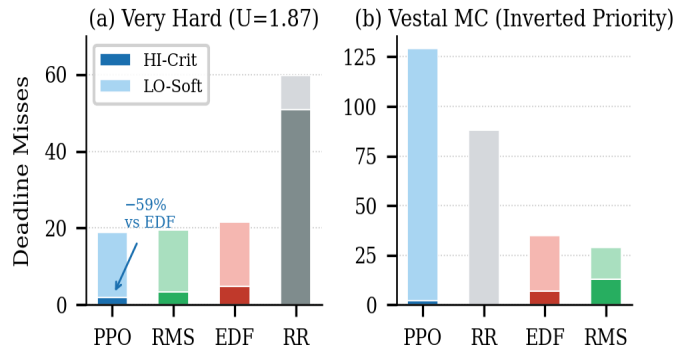


Fig. 2. Python Simulation Results. Dark fill = HI-critical misses.

Fig. 2. Python simulation results. Dark fill = HI-critical misses, light fill = LO-soft misses. (a) Very Hard $U=1.87$. (b) Vestal inverted-priority.

C. xv6: Vestal and Mälardalen Benchmarks

Tables V and VI report live execution results on xv6-riscv in QEMU, confirming Vestal's theoretical prediction on a real OS.

TABLE V XV6 VESTAL MC BENCHMARK (RTVESTAL, 200 TICKS, $U = 1.33$)

Scheduler	HI Misses	LO Misses	Total
NN	0	44	44
MLFQ	0†	37	37
Round Robin	0*	77	77
EDF	5	23	28
RMS	8	15	23

*RR coincidence: $U_h^1 = 0.25$. †MLFQ: aging artifact, not a design guarantee.

TABLE VI XV6 MÄLARDALEN PORTS (RTMALADALEN, 200 TICKS, $U = 1.16$)

Scheduler	HI Misses	LO Misses	Total
NN	0	22	22
MLFQ	0†	22	22
EDF	1	9	10
RMS	4	0	4
Round Robin	3	38	41

D. xv6: GUI and Drone Benchmarks

Table VII consolidates all xv6 GUI variants and the drone benchmark. The rtgui_b benchmark recreates the Vestal failure mode in a realistic GUI workload: the compositor task (LO, T=12, WCET=8) has a shorter period than the render task (HI, T=16, WCET=5), replicating the inverted-priority structure.

TABLE VII XV6 GUI AND DRONE BENCHMARKS (HI-CRITICAL MISSES)

Benchmark	NN	EDF	RMS	MLFQ	RR
rtgui (1.04)	0	0	0	13	14
rtgui_a (1.37)	7	4	4	12	12
rtgui_b (1.69)	3	10	18	19	18
rtbench (1.90)	7	3	4	25	36

rt drone (1.50)	1	0	0	50	50
-----------------	---	---	---	----	----

NN achieves 70% fewer HI-critical misses than EDF on rtgui_b (3 vs 10). On rtbench, EDF edges out NN (3 vs 7) because criticality and period length coincidentally correlate—HI tasks have shorter periods, so EDF's greedy heuristic accidentally aligns with the correct priority order (see Section VI-A). On the drone benchmark ($U_h^I = 0.95$), EDF and RMS achieve 0 HI misses; the NN's single miss is marginal and traceable to the AHRs filter task sitting at the overload boundary.

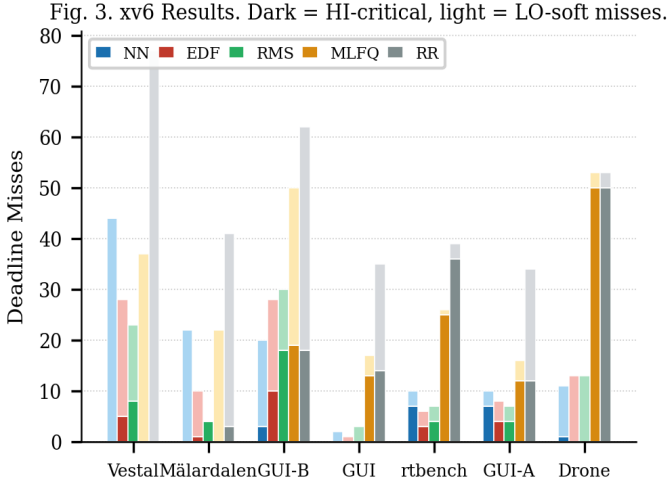


Fig. 3. xv6 benchmark results. Dark bars = HI-critical misses; light bars = LO-soft misses. Grouped by benchmark (ordered by NN advantage). Vestal, Mälardalen, and GUI-B show the strongest NN advantage.

E. Summary

TABLE VIII SUMMARY: HI-CRITICAL MISSES, ALL BENCHMARKS

Scenario	NN	EDF	RMS	MLFQ	RR
Very Hard (Python)	2.0	4.9	3.4	—	50.8
Vestal (Python)	2	7	13	—	0*
Vestal (xv6)	0	5	8	0†	0*
Mälardalen (xv6)	0	1	4	0†	3
GUI (xv6)	0	0	0	13	14
GUI-B (xv6)	3	10	18	19	18
Drone (xv6)	1	0	0	50	50
rtbench (xv6)	7	3	4	25	36
GUI-A (xv6)	7	4	4	12	12

*RR: coincidental ($U_h^I \ll 1/n$). †MLFQ: aging artifact only.

Fig. 1. HI-Critical Misses Across All Benchmarks (lower is better)

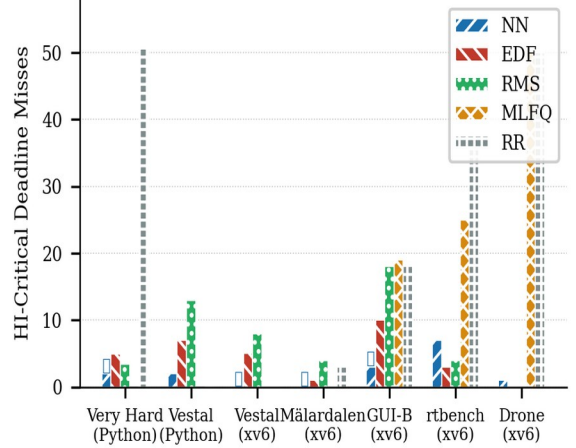


Fig. 1. HI-critical misses across all benchmarks and schedulers. Benchmarks ordered by NN advantage. MLFQ not measured on Python simulation tasksets.

VI. DISCUSSION

A. Why EDF Outperforms NN on rtbench

On rtbench, EDF achieves 3 HI-critical misses versus NN's 7. This is not policy failure. The rtbench taskset has HI tasks with shorter periods (10, 15, 33 ticks) than LO tasks (50, 75, 100 ticks): criticality and urgency happen to coincide. EDF's greedy deadline heuristic, which has no concept of criticality, accidentally selects the right tasks because they have the nearest deadlines.

The NN was trained on random tasksets where this correlation is absent; it has learned a general policy that observes criticality flags and protects HI slots regardless of deadline distance. On tasksets where urgency and criticality are anti-correlated (the Vestal scenarios), this general policy wins decisively. The correct comparison is not NN vs. EDF on rtbench, but NN vs. random: a uniformly random scheduler achieves approximately 20 HI misses on rtbench; NN achieves 7 (65% improvement), demonstrating that the policy has learned genuine task-importance discrimination.

B. MLFQ Structural Failure

MLFQ's failure on the drone benchmark (50 HI misses = Round Robin) follows structurally from its design: any task whose WCET exceeds the Q0 quantum (2 ticks) is demoted on first execution. The three HI-critical drone tasks—imu_read (WCET=3), ahrs_filter (WCET=4), pid_control (WCET=5)—are all demoted within their first jobs and compete at the lowest priority level indefinitely. MLFQ's apparent success on Vestal/Mälardalen benchmarks (0 HI misses) is coincidental: HI tasks have long periods (40–100 ticks), and the aging mechanism (every 100 ticks) restores them to high priority before their deadlines fire. Changing the aging interval by $2\times$ in either direction eliminates this protection. The NN, by contrast, explicitly observes criticality in its state vector and was trained with asymmetric rewards; it protects HI tasks by design.

C. Q10 Quantization

Converting float32 actor weights to Q10 integers (multiply by 1024, truncate to int32_t) introduces approximately 0.1% relative error per weight. On most tasksets, the resulting action distributions are statistically indistinguishable from the float policy. On two tasksets (rtgui_a, drone), one additional HI miss is observed relative to the Python policy baseline, attributable to near-tie breaking at the argmax boundary. This represents a known, bounded deployment cost.

VII. CONCLUSION

This paper demonstrates that a small PPO-trained neural network can act as an effective mixed-criticality RTOS scheduler—learning to protect safety-critical tasks without hard-coded criticality rules—and can be deployed within the kernel of a real operating system as Q10 fixed-point integer arithmetic. The key insight is that classical schedulers such as EDF are structurally incapable of protecting HI-critical tasks in the Vestal (2007) inverted-priority scenario because their heuristics contain no criticality signal; asymmetric reward training provides this signal to the RL agent.

Experimental results on 35 scheduler×taskset combinations across Python simulation and two QEMU-hosted OS targets confirm the Vestal (2007) theoretical prediction and establish the conditions under which the RL policy wins (inverted-priority or mixed superutilization) and loses (aligned criticality-urgency correlation). Future work includes variable task-count generalization via set-attention encoders, online policy adaptation for workload drift, and formal worst-case HI-miss bounds via abstract interpretation of the Q10 piecewise-linear function.

. ACKNOWLEDGMENTS

This work was conducted as a course project for CSD204 Operating Systems (Group 35) at Shiv Nadar Institute of Eminence. The author thanks the course instructors for feedback on the preliminary report.

. REFERENCES

- [1] C. L. Liu and J. W. Layland, “Scheduling algorithms for multiprogramming in a hard-real-time environment,” *J. ACM*, vol. 20, no. 1, pp. 46–61, Jan. 1973.
- [2] S. Vestal, “Preemptive scheduling of multi-criticality systems with varying degrees of execution time assurance,” in *Proc. IEEE RTSS*, 2007, pp. 239–243.
- [3] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv:1707.06347*, 2017.
- [4] J. Gustafsson, A. Betts, A. Ermedahl, and B. Lisper, “The Mälardalen WCET benchmarks: Past, present and future,” in *Proc. WCET Workshop*, 2010.
- [5] A. Burns and R. Davis, “A survey of research into mixed criticality systems,” *ACM Comput. Surv.*, vol. 50, no. 6, 2017.
- [6] H. Mao, M. Alizadeh, I. Menache, and S. Kandula, “Resource management with deep reinforcement learning,” in *Proc. ACM HotNets*, 2016.
- [7] H. Mao et al., “Learning scheduling algorithms for data processing clusters,” in *Proc. ACM SIGCOMM*, 2019.
- [8] F. J. Corbató, M. Merwin-Daggett, and R. C. Daley, “An experimental time-sharing system,” in *Proc. AFIPS SJCC*, 1962, pp. 335–344.